

Matplotlib: Visualización de Datos en Python

Guía Rápida e Infografía Didáctica basada en el Cuaderno de Clase

1. Conceptos Fundamentales

La visualización de datos es la representación gráfica de información para identificar patrones y tendencias.

Convención de Importación:

```
import matplotlib.pyplot as plt
import numpy as np
```

2. Gráfico de Línea Simple

Ideal para graficar funciones matemáticas como $f(x) = x^2$.

```
# Generar datos secuenciales
x = np.arange(-5, 6, 1)
y = x**2

# Graficar básico
plt.plot(x, y)
plt.show()
```

3. API Orientada a Objetos

Permite un control total y explícito de la figura estructurando el lienzo con **Figure** y **Axes**.

```
fig, ax = plt.subplots()
ax.plot(x, y)
plt.show()
```

Uso de Subplots (Lienzos Múltiples)

```
# Matriz de 1 fila y 2 columnas
fig, ax = plt.subplots(1, 2)
ax[0].plot(x, x**2)
ax[1].plot(x, x**3)
plt.show()
```

5. Personalización Avanzada

Agrega contexto científico y claridad visual a tus representaciones métricas:

• Títulos y Etiquetas:

```
plt.title("Gráfica de f(x)")
plt.xlabel("Eje X") / ylabel()
```

• Texto Anotado:

```
plt.text(x_pos, y_pos, "Nota")
```

• Líneas Guía Relevantes:

```
plt.hlines(y, xmin, xmax,
linestyle='dashed')
plt.vlines(x, ymin, ymax,
linestyle='dashed')
```

• Cuadrícula de Soporte:

```
plt.grid(True)
```

• Límites y Ticks:

```
plt.xlim(min, max)
plt.xticks(np.arange(-5, 6, 1))
```

6. Estilización de Marcadores y Líneas

Personaliza las propiedades completas de la función `plt.plot()`:

```
plt.plot(x, y,
linestyle="--",
marker="o",
color="red",
markersize=10,
markerfacecolor="blue",
linewidth=4)
```

Estilos - continuo, -- cortado, : punteado.

7. Más Gráficos

Scatter Plot (Dispersión)

```
plt.scatter(x, y, color='blue')
```

Pie Chart (Gráfico Circular)

```
sizes = [90, 180, 30, 60]
labels = ["A", "B", "C", "D"]
plt.pie(sizes, labels=labels)
```

4. Tipos Básicos de Gráficos

Gráfico de Barras (Categorías)

```
cats = ["A", "B", "C", "D"]  
vals = [10, 15, 7, 12]  
plt.bar(cats, vals)
```

Histogramas (Distribución de frecuencias)

```
data = np.random.randn(1000)  
plt.hist(data, bins=10)
```

8. Exportación de Resultados

Guarda tus creaciones con calidad vectorial de manera local:

```
# Guardar en formato PDF  
plt.savefig("grafico.pdf")  
  
# Guardar en formato JPEG  
plt.savefig("grafico.jpg")
```